TP4 - Intro à la statistique spatiale 2A - autocorrélation spatiale - corrigé

OBJECTIFS DU TP :

- Le but de ce TP est de s'iniitier à l'étude de l'autocorrélation spatiale sur données surfaciques.
- Afin d'utiliser une version de R plus récente (et une version des packages les plus récentes aussi), vous travaillerez sur le datalab (plateforme du sspcloud, service de l'Insee) : https://datalab.sspcloud.fr.

Les packages nécessaires pour ce TP sont listés ci-dessous :

```
# Chargement des packages
library(dplyr)
library(sf)
library(spdep)
library(leaflet)
library(RColorBrewer)
```

Exercice 1

Nous allons étudier s'il existe un phénomène d'autocorrélation spatiale des revenus médians par iris marseillais.

Vous utiliserez le fond des communes des Iris France entiere ainsi que les données.

1. Commencez par vous créer votre jeu de données : sélectionnez la ville de Marseille sur votre fond d'Iris et faire la jointure avec votre jeu de données.

```
# Import des données
iris <- st_read("./iris_franceentiere_2021/iris_franceentiere_2021.shp", options = "ENCODING=UTF8")
```

```
options:        ENCODING=UTF8
Reading layer `iris_franceentiere_2021' from data source
  `/home/onyxia/work/2A_stat_spatiale/2025-2026/TP4/iris_franceentiere_2021/iris_franceentiere_2021.s
  using driver `ESRI Shapefile'
```

```
Warning in CPL_read_ogr(dsn, layer, query, as.character(options), quiet, : GDAL
Message 1: One or several characters couldn't be converted correctly from UTF8
to UTF-8.  This warning will not be emitted anymore
```

```
Simple feature collection with 16351 features and 3 fields
Geometry type: MULTIPOLYGON
Dimension:     XY
Bounding box:  xmin: -63.15332 ymin: -21.38963 xmax: 55.83665 ymax: 51.0668
Geodetic CRS:  WGS 84
```

```
data <- read.csv2("./data/BASE_TD_FILO_DISP_IRIS_2018.csv", sep=";", dec=".", encoding = "utf-8")

# Jointure
marseille <- iris %>%
  filter(substr(depcom,1,3)=="132") %>%  # iris de Marseille (132XXXXXX)
  left_join(data %>% select(code=IRIS,DISP_MED18), # on renomme la variable data$IRIS
            by="code")
```

2. Le système de projection de votre table est en WGS84, convertissez-le en Lambert-93 (EPSG 2154).

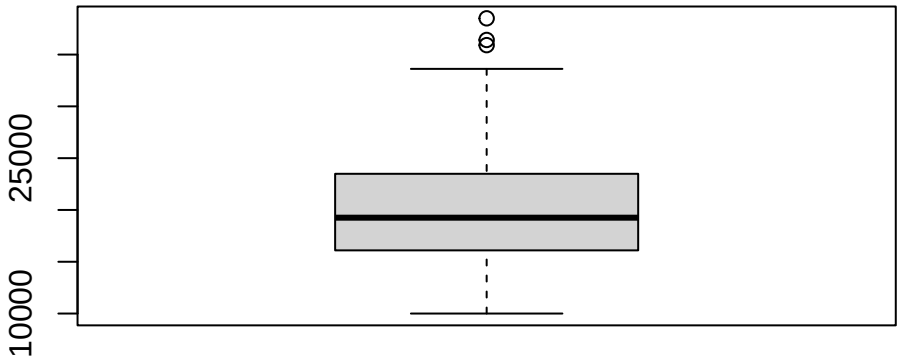
```r
marseille <- marseille %>%
  st_transform(2154) # code EPSG 2156 du Lambert-93
```

3. Faites un premier résumé statistique de la variable de revenu médian. Faites également un boxplot du revenu moyen en fonction des arrondissements.

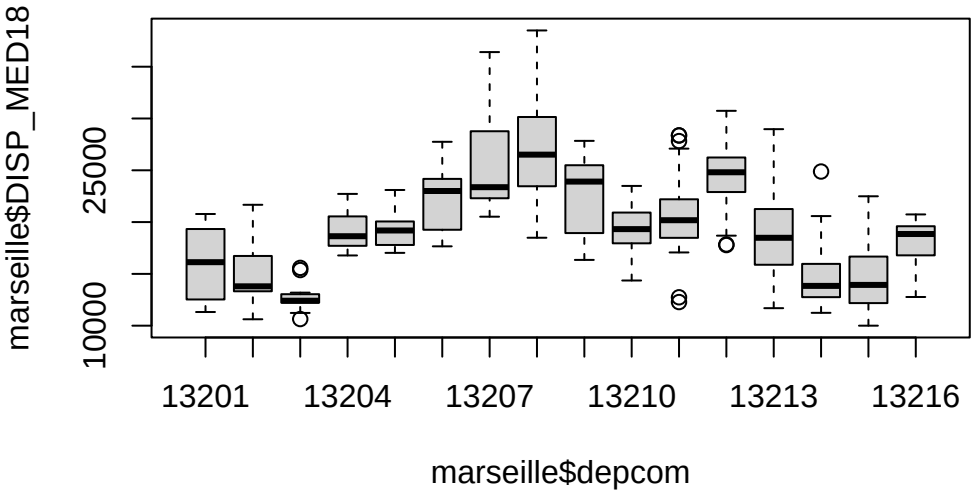
```r
summary(marseille$DISP_MED18) # 393 iris MRS - 57 NA sur DISP_MED18 = 336 iris
```

```
   Min. 1st Qu.  Median    Mean 3rd Qu.    Max.    NA's 
  10000   16115   19255   19767   23490   38500      57 
```

```r
boxplot(marseille$DISP_MED18)
```




```r
boxplot(marseille$DISP_MED18 ~ marseille$depcom)
```



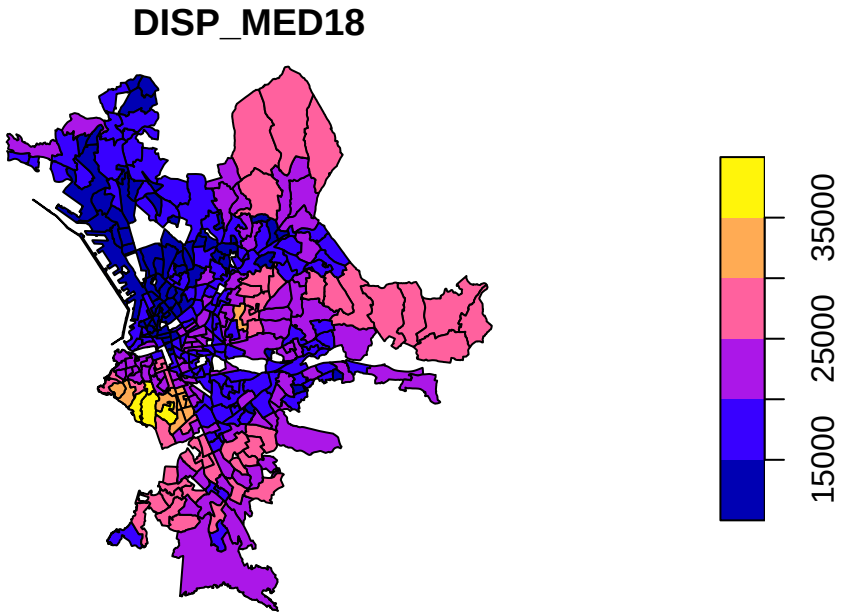

```r
# Pour aller plus loin, analyse de la variance/test de Fisher (ANOVA à 1 facteur, = depcom) :
# (H0) : mu_1 = mu_2 = ... = m_16 (ie. les K=16 arrondissements ont des moyennes
#                                  de revenus médians par iris égales, ie pas d'influence
#                                  de l'arrondissement sur le revenu médian)
# vs. (H1) : il existe i!=j tq mu_i != mu_j
```

```r
# REM : on suppose que depcom influence uniquement la moyenne des obs et non leur
#  dispersion ie. on fait l'hyp (nécessaire pour le test) que chacun des K=16 éch
#  (constitués de n_k iris) est issu d'une N(mu_k ; sigma²) où 1 <= k <= 16 = K.
# Stat de test : rapport de Variance INTER/INTRA * (n-K/K-1) ~ F(K-1;n-K) sous (H0)
# pvaleur du test de fisher ~ 0 --> rejet de (H0) l'égalité des moyennes au risques 5 % ie.
# la différence des revenus disponibles médians par arrondissement est signficative.
summary(aov(DISP_MED18 ~ depcom, data = marseille))
```

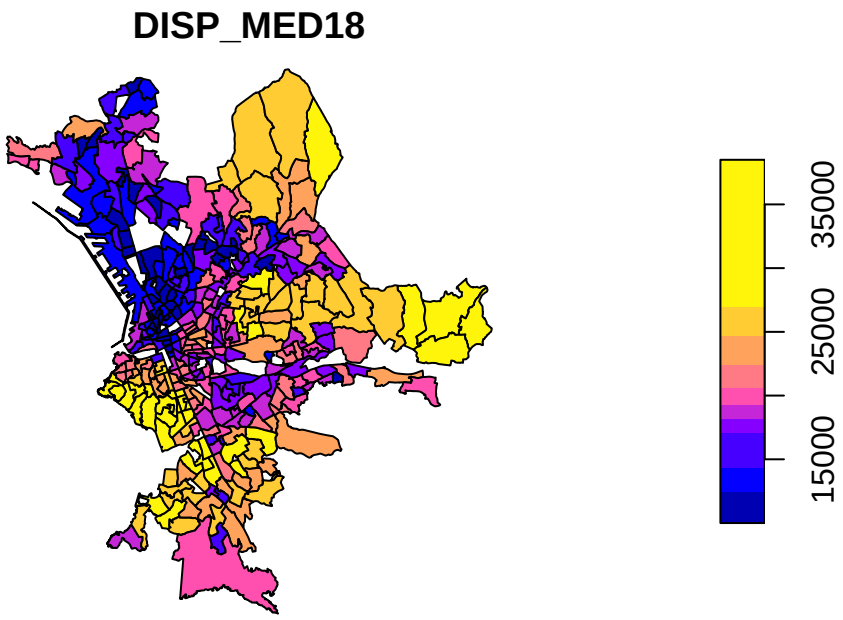
```
              Df    Sum Sq   Mean Sq F value Pr(>F)
depcom        15 5.759e+09 383941516    30.7 <2e-16 ***
Residuals    320 4.003e+09  12508008
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
57 observations deleted due to missingness
```

4. Supprimer les valeurs manquantes puis représenter la carte de Marseille en fonction des revenus. Vous pouvez utiliser la fonction `plot()` (n'hésitez pas à utiliser une ou plusieurs méthodes de discrétisation automatique - argument `breaks`). Au vu de la carte, vous semble-t-il y avoir un phénomène spatial dans la distribution des revenus ?

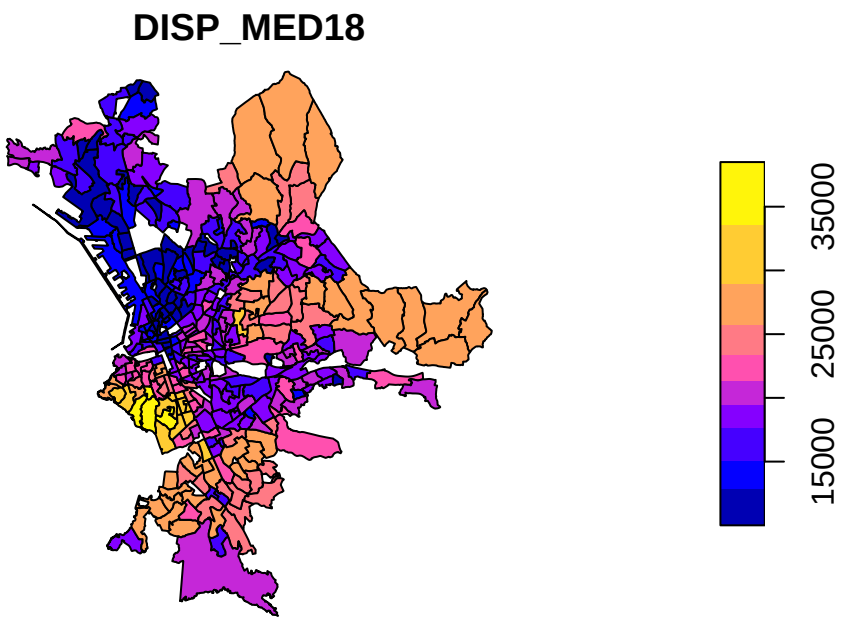
```r
marseille<-marseille %>%
  filter(!is.na(DISP_MED18))
  # tidyr::drop_na() # autre façon de retirer les valeurs manquantes

plot(marseille["DISP_MED18"]) # analyse continue
```




```r
# analyse avec discrétisation automatique :
plot(marseille["DISP_MED18"], breaks = "quantile", nbreaks = 10)
```




```
plot(marseille["DISP_MED18"], breaks = "jenks") # idem automatique
```




```
library(mapview)

mapview(
  marseille,
  z=c("DISP_MED18"),
  alpha.regions=0.35,
  layer.name="Revenu_median",
  label="libelle"
)
```

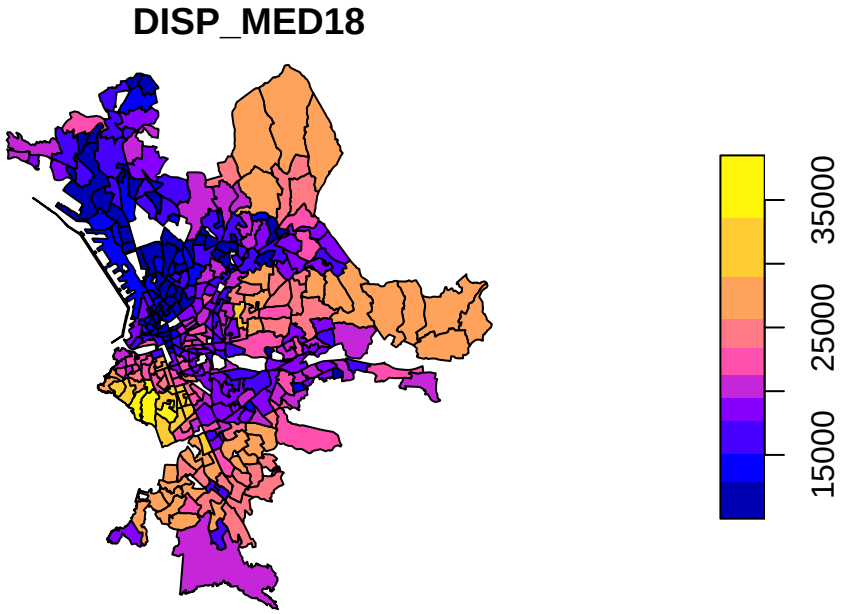
5. Pour nous faire une première idée de la dimension spatiale de la distribution des revenus, nous allons représenter les mêmes revenus mais distribués de manière aléatoire au sein des iris marseillais. On pourra ainsi comparer la carte de la distribution réelle des revenus avec la carte de la distribution aléatoire. Pour cela,

a. créez une permutation aléatoire des revenus disponibles médians par iris avec la fonction `sample()` et à partir de la variable `DISP_MED18` du fond des iris de Marseille. Vous stockerez ce vecteur dans une nouvelle variable du fond des iris nommée `DISP_MED18_ALEA`.

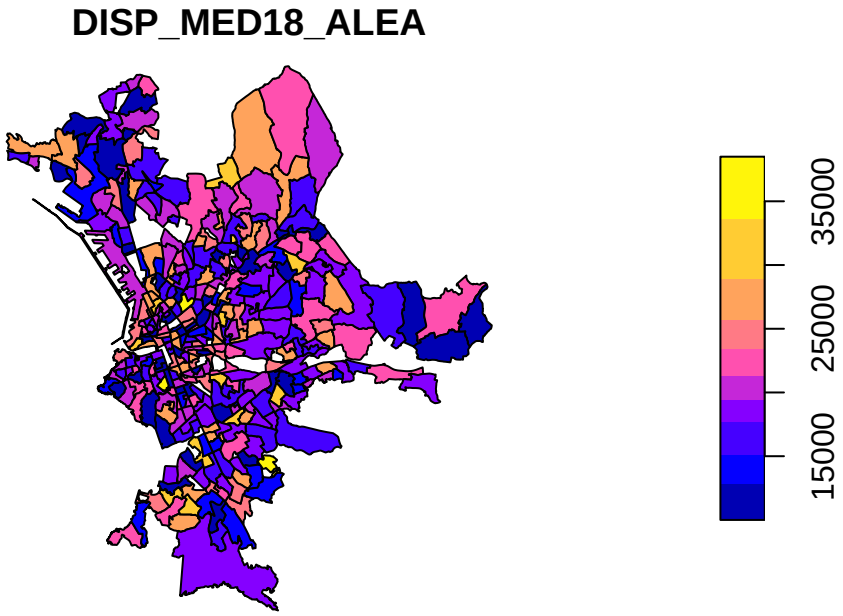
```r
set.seed(1993)
marseille <- marseille %>%
  mutate(DISP_MED18_ALEA = sample(DISP_MED18))
```

b. représentez sur une carte la distribution géographique de la variable que vous venez de créer. Comparez le résultat avec la carte réalisée sur la distribution réelle. La distribution spatiale réelle des revenus est-elle proche de la distribution aléatoire ?

```r
plot(marseille["DISP_MED18"], breaks = "jenks")
```




```r
plot(marseille["DISP_MED18_ALEA"], breaks = "jenks")
```




```r
# La comparaison des deux cartes semble suggérer que la carte représentant la distribution
#  réelle des revenus est très différente de la carte d'une distribution aléatoire.
# Le phénomène semble spatialement corrélé.
```

6. Pour corroborer la conclusion du 5., nous allons mesurer et tester le phénomène d'autocorrélation spatiale.

Un phénomène est autocorrélé spatialement quand la valeur de la variable étudiée à un endroit donné est

plus liée aux valeurs de ses voisins plutôt qu'à celles des autres (non-voisins). On parle d'*autocorrélation positive* si des voisins ont tendance à prendre des valeurs similaires et d'*autocorrélation négative* si des voisins ont tendance à prendre des valeurs différentes.

a. Quel type d'autocorrélation spatiale, le phénomène étudié semble-t-il avoir ?

```
# Autocorrélation positive. La carte montre en effet des zones regroupant des iris à revenus faibles
#  (centre et nord de Marseille) et des iris à forts revenus (sud du Vieux Port par exemple).
```

b. Pour étudier le phénomène, il nous faut construire une matrice de voisinage. Il existe plusieurs façons de définir le voisinage de nos iris. Dans un premier temps, nous allons définir le voisinage par la contiguïté : deux iris sont voisins s'ils sont contigus.

Pour limiter la taille des objets créés, nous allons travailler avec des listes plutôt qu'avec des matrices carrées.

Extraire la liste des voisins de chaque Iris. Pour cela, vous utiliserez la fonction `spdep::poly2nb()`. Par défaut, il s'agit de la contiguité dite `QUEEN` qui reprend les mouvements de la Reine aux échecs. Prenez connaissance de l'objet créé et réalisez un résumé de l'objet en sortie avec la fonction `summary()`.

```r
voisins <- poly2nb(marseille) #par défaut: queen = TRUE
str(voisins)
```

```
List of 336
 $ : int [1:6] 2 3 8 19 23 25
 $ : int [1:5] 1 3 5 8 15
 $ : int [1:6] 1 2 4 5 7 19
 $ : int [1:4] 3 7 19 37
 $ : int [1:7] 2 3 6 7 8 15 16
 $ : int [1:8] 5 7 11 12 13 15 16 70
 $ : int [1:6] 3 4 5 6 13 14
 $ : int [1:6] 1 2 5 9 15 90
 $ : int [1:3] 8 90 94
 $ : int [1:5] 23 91 93 114 115
 $ : int [1:6] 6 12 16 63 64 70
 $ : int [1:5] 6 11 13 14 64
 $ : int [1:4] 6 7 12 14
 $ : int [1:8] 7 12 13 31 32 35 60 64
 $ : int [1:7] 2 5 6 8 16 72 90
 $ : int [1:6] 5 6 11 15 70 72
 $ : int [1:8] 26 42 45 46 306 308 310 311
 $ : int [1:4] 20 28 36 37
 $ : int [1:8] 1 3 4 20 21 22 25 37
 $ : int [1:6] 18 19 22 28 29 37
 $ : int [1:4] 19 22 25 26
 $ : int [1:5] 19 20 21 26 29
 $ : int [1:4] 1 10 24 25
 $ : int [1:3] 23 25 26
 $ : int [1:6] 1 19 21 23 24 26
 $ : int [1:9] 17 21 22 24 25 27 28 29 45
 $ : int [1:4] 26 28 36 45
 $ : int [1:7] 18 20 26 27 29 36 45
 $ : int [1:4] 20 22 26 28
 $ : int [1:6] 31 34 59 60 285 298
 $ : int [1:6] 14 30 32 34 59 60
 $ : int [1:6] 14 31 33 34 35 38
 $ : int [1:6] 32 34 38 39 40 44
 $ : int [1:6] 30 31 32 33 39 285
 $ : int [1:4] 14 32 37 38
 $ : int [1:7] 18 27 28 37 38 44 45
 $ : int [1:7] 4 18 19 20 35 36 38
 $ : int [1:7] 32 33 35 36 37 40 44
 $ : int [1:5] 33 34 40 282 285
 $ : int [1:8] 33 38 39 41 42 44 46 282
 $ : int [1:7] 40 42 43 282 286 287 288
```

```
$ : int [1:9] 17 40 41 43 45 46 287 288 311
$ : int [1:2] 41 42
$ : int [1:6] 33 36 38 40 45 46
$ : int [1:8] 17 26 27 28 36 42 44 46
$ : int [1:5] 17 40 42 44 45
$ : int [1:9] 49 51 52 79 81 234 235 236 238
$ : int [1:5] 49 61 62 73 79
$ : int [1:6] 47 48 50 51 61 79
$ : int [1:4] 49 51 53 61
$ : int [1:5] 47 49 50 52 53
$ : int [1:5] 47 51 53 227 234
$ : int [1:8] 50 51 52 54 55 56 61 227
$ : int [1:5] 53 55 56 227 274
$ : int [1:6] 53 54 56 57 61 64
$ : int [1:5] 53 54 55 57 58
$ : int [1:6] 55 56 58 59 60 64
$ : int [1:4] 56 57 59 273
$ : int [1:8] 30 31 57 58 60 271 273 298
$ : int [1:6] 14 30 31 57 59 64
$ : int [1:8] 48 49 50 53 55 62 63 64
$ : int [1:4] 48 61 63 69
$ : int [1:6] 11 61 62 64 69 70
$ : int [1:8] 11 12 14 55 57 60 61 63
$ : int [1:5] 66 68 85 86 181
$ : int [1:5] 65 67 68 181 198
$ : int [1:5] 66 73 75 80 198
$ : int [1:5] 65 66 78 86 87
$ : int [1:5] 62 63 70 71 73
$ : int [1:7] 6 11 16 63 69 71 72
$ : int [1:5] 69 70 72 73 74
$ : int [1:6] 15 16 70 71 74 90
$ : int [1:8] 48 67 69 71 74 75 79 80
$ : int [1:7] 71 72 73 75 76 89 90
$ : int [1:4] 67 73 74 76
$ : int [1:4] 74 75 77 89
$ : int [1:3] 76 78 89
$ : int [1:5] 68 77 86 87 89
$ : int [1:6] 47 48 49 73 80 81
$ : int [1:5] 67 73 79 81 198
$ : int [1:4] 47 79 80 238
$ : int [1:7] 83 84 85 95 120 123 131
$ : int [1:3] 82 84 95
$ : int [1:9] 82 83 85 86 87 88 92 94 95
$ : int [1:6] 65 82 84 86 131 181
$ : int [1:6] 65 68 78 84 85 87
$ : int [1:7] 68 78 84 86 88 89 94
$ : int [1:5] 84 87 89 90 94
$ : int [1:7] 74 76 77 78 87 88 90
$ : int [1:8] 8 9 15 72 74 88 89 94
$ : int [1:3] 10 92 93
$ : int [1:6] 84 91 93 94 95 96
$ : int [1:7] 10 91 92 96 97 113 114
$ : int [1:7] 9 84 87 88 90 92 95
$ : int [1:9] 82 83 84 92 94 96 97 98 123
$ : int [1:5] 92 93 95 97 98
$ : int [1:9] 93 95 96 98 107 108 113 123 125
$ : int [1:3] 95 96 97
$ : int [1:7] 100 101 102 103 108 110 112
 [list output truncated]
- attr(*, "class")= chr "nb"
- attr(*, "region.id")= chr [1:336] "1" "2" "3" "4" ...
- attr(*, "call")= language poly2nb(pl = marseille)
- attr(*, "type")= chr "queen"
```

```
 - attr(*, "snap")= num 0.01
 - attr(*, "sym")= logi TRUE
 - attr(*, "ncomp")=List of 2
  ..$ nc     : int 1
  ..$ comp.id: int [1:336] 1 1 1 1 1 1 1 1 1 1 ...
```

```r
summary(voisins)
```

```
Neighbour list object:
Number of regions: 336
Number of nonzero links: 1784
Percentage nonzero weights: 1.580215
Average number of links: 5.309524
Link number distribution:

 1  2  3  4  5  6  7  8  9 10
 4 14 32 64 73 70 37 25 14  3
4 least connected regions:
118 203 204 280 with 1 link
3 most connected regions:
208 222 264 with 10 links
```

c. Combien de voisins a le quatrième élément de la liste ?

```r
voisins[[4]] # 4
```

```
[1]  3  7 19 37
```

7. Nous allons transformer la matrice de contiguité en une matrice de pondérations. L'idée est d'affecter un poids identique à chacun des voisins d'un iris.

a. Créez une liste de poids à partir de la liste de voisins précédemment créée. Pour cela, utilisez la fonction `spdep::nb2listw()`, avec l'argument `zero.policy=TRUE` pour intégrer les Iris n'ayant potentiellement pas de voisins (par défaut, la fonction exclut les observations sans voisin).

```r
ponderation <- nb2listw(voisins, zero.policy = TRUE)
```

b. Prenez connaissance de l'objet créé avec la fonction `str()` et l'argument `max.level = 1` et réalisant un résumé de la liste avec la fonction `summary()`.

```r
str(ponderation, max.level = 1)
```

```
List of 3
 $ style     : chr "W"
 $ neighbours:List of 336
  ..- attr(*, "class")= chr "nb"
  ..- attr(*, "region.id")= chr [1:336] "1" "2" "3" "4" ...
  ..- attr(*, "call")= language poly2nb(pl = marseille)
  ..- attr(*, "type")= chr "queen"
  ..- attr(*, "snap")= num 0.01
  ..- attr(*, "sym")= logi TRUE
  ..- attr(*, "ncomp")=List of 2
 $ weights   :List of 336
  ..- attr(*, "mode")= chr "binary"
  ..- attr(*, "W")= logi TRUE
  ..- attr(*, "comp")=List of 1
 - attr(*, "class")= chr [1:2] "listw" "nb"
 - attr(*, "region.id")= chr [1:336] "1" "2" "3" "4" ...
 - attr(*, "call")= language nb2listw(neighbours = voisins, zero.policy = TRUE)
 - attr(*, "zero.policy")= logi TRUE
```

```r
summary(ponderation) # zero.policy=TRUE
```

```
Characteristics of weights list object:
Neighbour list object:
Number of regions: 336
```

```
Number of nonzero links: 1784
Percentage nonzero weights: 1.580215
Average number of links: 5.309524
Link number distribution:

 1  2  3  4  5  6  7  8  9 10
 4 14 32 64 73 70 37 25 14  3
4 least connected regions:
118 203 204 280 with 1 link
3 most connected regions:
208 222 264 with 10 links

Weights style: W
Weights constants summary:
    n     nn  S0       S1       S2
W 336 112896 336 139.1824 1392.104
```

```
# Il s'agit d'une liste de trois éléments :

# -   style = type de matrice (W =\> weigths)
# -   neighbours = liste des voisins (par exemple `ponderation$neighbours[[1]]`
#     fournit la liste des voisins de la première observation)
# -   weights = liste des poids (par exemple `ponderation$weights[[1]]` fournit
#     la liste des poids des voisins de la première observation)
```

b. Vérifiez que la somme des pondérations associées à chaque pondération est égale à 1.

```
all(sapply(ponderation$weights, sum) == 1)
```

```
[1] TRUE
```

8. Une autre façon très visuelle de vérifier la présence d'une autocorrélation est de dresser le diagramme de Moran. La matrice de pondération calculée en 7. va nous permettre de le calculer.

a. Créer une variable des revenus disponibles centrés réduits avec la fonction `scale()`. Vous la nommerez `DISP_MED18_STD` et l'ajouterez au fond des iris de Marseille. Vous vérifierez que la nouvelle variable est bien centrée (moyenne = 0) et réduite (SD = 1).

```
marseille <- marseille %>%
  mutate(DISP_MED18_STD = scale(DISP_MED18)) # centrer réduire la var DISP_MED18

mean(marseille$DISP_MED18_STD) # ~ 0
```

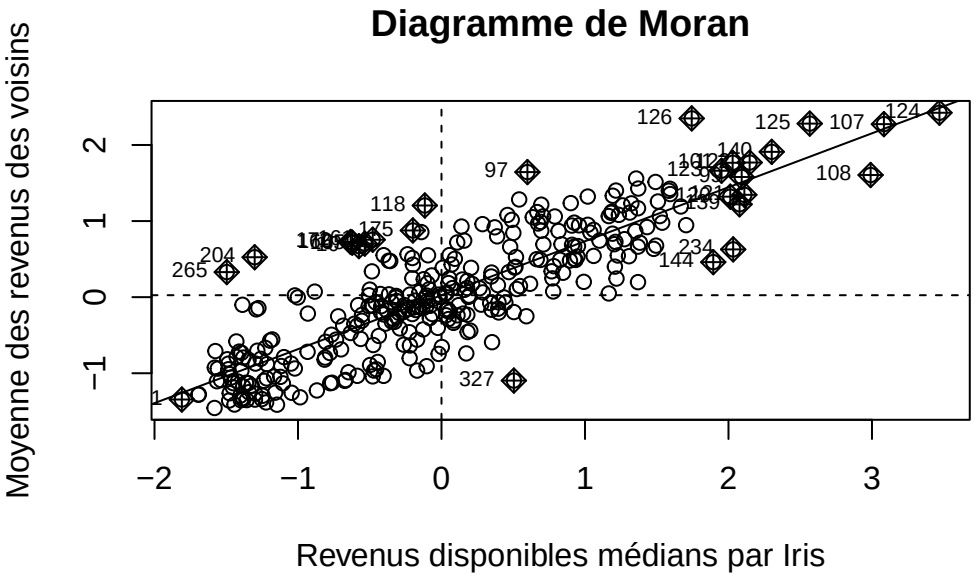
```
[1] -2.910483e-16
```

```
sd(marseille$DISP_MED18_STD) # 1
```

```
[1] 1
```

b. Dresser le diagramme de Moran avec la fonction `moran.plot()` à partir de la variable standardisée (utiliser la fonction `as.numeric()` si un problème apparaît). Le second argument à préciser (`listw`) correspond à la liste des poids des voisins que vous avez créée précédemment.

```
moran.plot(
  as.numeric(marseille$DISP_MED18_STD),
  listw = ponderation,
  xlab = "Revenus disponibles médians par Iris",
  ylab = "Moyenne des revenus des voisins",
  main = "Diagramme de Moran"
)
```



c. Le diagramme de Moran représente, pour chaque observation (ici un Iris), croise deux informations :

— en abscisse, est représenté le revenu médian disponible observé au sein de l'iris (variable centrée réduite) ;
— en ordonnées, est représentée la moyenne des revenus médians des voisins de l'iris observé.

Interprétez les quatre cadrans du diagramme.

```
# -   cadran en haut à droite : `high-high` = iris aux revenus médians élevés et
#                                dont les voisins ont également des revenus élevés;
# -   cadran en bas à droite : `high-low` = iris aux revenus médians élevés entourés
#                               de voisins à faibles revenus;
# -   cadran en bas à gauche : `low-low` = iris aux faibles revenus médians et entourés
#                               de voisins également à faibles revenus;
# -   cadran en haut à gauche : `low-high` = iris aux faibles revenus entourés
#                               de voisins aux revenus élevés.
```

d. D'après le diagramme de Moran, les revenus médians semblent-ils autocorrélés spatialement ? Si oui, l'autocorrélation vous semble-t-elle positive ou négative ?

```
# Les observations sont relativement bien alignées selon la première diagonale du plan.
# Le phénomène semble bien autocorrélé spatialement et cette autocorrélation semble positive.
# En effet, les iris à hauts revenus sont plutôt entourés de voisins à hauts revenus et les iris
#  à faibles revenus sont plutôt entourés de voisins à faibles revenus.
```

9. Il existe une mesure globale de l'autocorrélation spatiale d'un phénomène. Il s'agit du **I de Moran**.

a. Calculez cet indice et sa significativité avec la fonction `spdep::moran.test()` utilisée de la façon suivante : `moran.test(marseille$DISP_MED18_STD, ponderation, randomisation = TRUE)`. Le dernier argument signifie que la distribution observée est comparée à une distribution aléatoire obtenue par permutation des valeurs observées.

```
moran.test(marseille$DISP_MED18_STD, # variable num centrée et réduite
           listw = ponderation,
           alternative = "greater", # argument par défaut, "two.sided"/"less"
           randomisation = TRUE)
```

```
    Moran I test under randomisation

data:  marseille$DISP_MED18_STD
weights: ponderation
```

```
Moran I statistic standard deviate = 20.44, p-value < 2.2e-16
alternative hypothesis: greater
sample estimates:
Moran I statistic       Expectation          Variance
      0.709225422      -0.002985075       0.001214128
```

b. Interpértez le résultat obtenu : confirme-t-il ou non votre hypothèse ?

```
# Le I de Moran vaut 0.709, ce qui est une valeur relativement élevée.
#  Le test réalisé est le suivant :
#
# -   H0 : le I de Moran est nul, cad le phénomène n'est pas spatialement autocorrélé.
# -   H1 : I \> 0 (par défaut -> on soupçonne une autoc. positive - cette alternative est préférable
#
# Ici, la p-valeur est très faible, conduisant à rejeter l'hypothèse nulle et donc à
#  conclure/confirmer l'autocorrélation spatiale positive au sein du phénomène.
#
# Attention, le I de Moran ne prend pas nécessairement ses valeurs entre -1 et 1.
#  Il est possible de calculer les bornes de l'intervalle de valeurs que I peut prendre.
#  Voir *Manuel d'Analyse Spatiale*,p. 62.
#
# Dans tous les cas, plus le I de Moran est grand, plus l'autocorrélation spatiale positive est forte
#
# > Remarque: le tp s'est concentré sur une seule définition du voisinage, entendu comme contiguïté.
#  Nous pourrions envisager que le voisinage d'une observation ne s'arrête pas qu'aux observations
#  partageant une frontière commune. En effet, nous pourrions considérer qu'est voisine d'une
#  observation donnée, toute observation située dans un rayon de X km. Dans ce cas, il faut définir
#  une distance (distance euclidienne à vol d'oiseau; distance en temps de parcours par la route;
#  k-plus-proches-voisins, etc.) pour définir l'éloignement des observations entre elles, et un
#  seuil délimitant le rayon de voisinage. En définissant ainsi le voisinage, il n'est plus
#  raisonnable d'attribuer le même poids d'influence à toutes les observations du voisinage :
#  on attribuera un poids plus fort aux voisins les plus proches. Classqiuement, le poids des
#  voisins est défini par l'inverse de la distance. Mais d'autres choix sont possibles.
#
# > Dans le cas des iris, la contiguïté semble la mesure la plus naturelle. En effet, leurs tailles
#  sont très différentes entre elles et il semble difficile de définir une distance et de choisir
#  un seuil adéquat pour tous. Par contre, si nous avions étudié le même phénomène sur des unités
#  spatiales homogènes en taille telles que des carreaux, la contiguïté aurait restreint le voisinage
#  à quelques carreaux et donc à une approche très localisée. Dans ce cas, un voisinage défini à part
#  d'une distance se serait révélé la meirlleure option.
#
# > Il est important de retenir que l'étude de votre phénomène sera nécessairement influencée par le
#  voisinage que vous choisirez. Cette influence sera plus ou moins importante.
# Pour aller plus loin, n'hésitez pas à consulter le *Manuel d'Analyse Spatiale*
```

10. BONUS - DECOUVRIR LES INDICATEURS D'AUTOCORRELATION LOCAUX.

L'indice de Moran est un indicateur global de mesure de l'autocorrélation. Mais, ce phénomène peut connaître une intensité très différente localement. Dans certains endroits de la ville de Marseille, la ressemblance des voisins peut être très forte et à d'autres endroits plus lâche. Des indicateurs locaux de mesure de l'autocorrélation spatiale sont nécessaires pour compléter l'analyse de la distribution spatiale des revenus disponibles médians à Marseille. Nous calculerons pour cela les ***LISA*** (*Local Indicators of Spatial Association*), ou ***I de Moran locaux***.

a. Calculez les Lisa avec la fonction `spdep::localmoran()` et stockez le résultat dans un objet appelé `mars_rev_lisa`.

```
mars_rev_lisa <- spdep::localmoran(marseille$DISP_MED18, # inutile d'utiliser la var _STD ici
                                   listw = ponderation, zero.policy = TRUE)
```

b. Etudiez l'objet obtenu, en utilisant notamment les fonctions `class()`, `str(.,max.level=1)` et `summary()`.

```
class(mars_rev_lisa)
```

```
[1] "localmoran" "matrix"     "array"
```

```
str(mars_rev_lisa, max.level=1)
```

```
 'localmoran' num [1:336, 1:5] 0.641 1.734 1.679 1.728 0.917 ...
 - attr(*, "dimnames")=List of 2
 - attr(*, "call")= language spdep::localmoran(x = marseille$DISP_MED18, listw = ponderation, zero.p
 - attr(*, "quadr")='data.frame':   336 obs. of  3 variables:
```

```
summary(mars_rev_lisa)
```

```
       Ii                E.Ii                Var.Ii               Z.Ii
 Min.   :-0.68520   Min.   :-3.606e-02   Min.   :9.400e-07   Min.   :-2.2000
 1st Qu.: 0.01926   1st Qu.:-4.794e-03   1st Qu.:1.665e-02   1st Qu.: 0.2344
 Median : 0.33538   Median :-1.414e-03   Median :9.219e-02   Median : 1.3037
 Mean   : 0.70923   Mean   :-2.985e-03   Mean   :2.225e-01   Mean   : 1.2811
 3rd Qu.: 1.12095   3rd Qu.:-2.597e-04   3rd Qu.:3.085e-01   3rd Qu.: 2.3493
 Max.   : 8.43570   Max.   :-1.700e-08   Max.   :2.893e+00   Max.   : 5.2108
 Pr(z != E(Ii))
 Min.   :1.900e-07
 1st Qu.:1.881e-02
 Median :1.459e-01
 Mean   :3.081e-01
 3rd Qu.:5.929e-01
 Max.   :9.963e-01
```

c. Quelle est la moyenne des indicateurs locaux (I_i) ?

```
mean(mars_rev_lisa[,1])
```

```
[1] 0.7092254
```

```
#ou
mean(mars_rev_lisa[,"Ii"])
```

```
[1] 0.7092254
```

```
# On remarque que la moyenne de Lisa est égale au I de Moran global. En effet, les Lisa
#  sont construits de telle sorte que leur somme soit proportionnelle au I de Moran global.
```

d. L'interprétation d'un Lisa est tout à fait similaire à l'indice global. Si l'indicateur local d'un Iris donné est positif, cela signifie qu'il est entouré d'Iris ayant des niveaux de revenus similaires. S'il est négatif, cela indique qu'il est plutôt entouré d'Iris ayant des niveaux de revenus différents (opposés).

Combien d'indicateurs locaux sont-ils négatifs ?

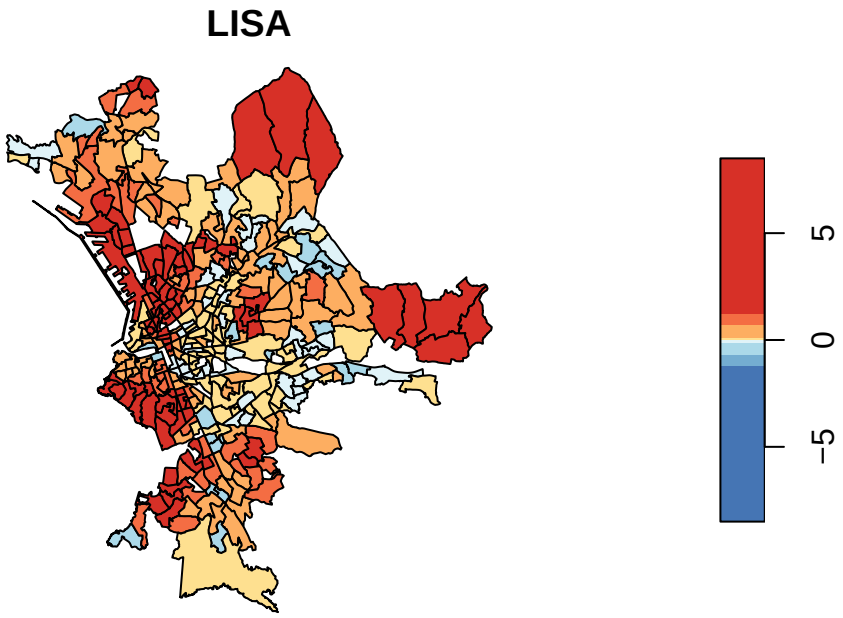
```r
table(mars_rev_lisa[,"Ii"] < 0)
```

```
FALSE  TRUE
  274    62
```

```r
# 62 Iris sur 336 ont un Lisa négatif. L'autocorrélation étudiée localement est donc bien
#  principalement positive : les Iris sont très majoritairement entourés d'Iris où
#  les revenus médians sont similaires.
```

e. Nous cherchons à représenter les Lisa sur une carte des Iris marseillais. Pour cela, ajouter les Lisa comme une nouvelle variable du fond des iris, variable que vous nommerez `LISA`. Vous utiliserez la palette suivante : `pal <- rev(RColorBrewer::brewer.pal(8, "RdYlBu"))` combinée avec les `breaks = c(-8.5,-1.2,-0.7,-0.1,0,0.1,0.7,1.2,8.5)`.

```r
marseille <- marseille %>%
  mutate(
    LISA = mars_rev_lisa[,"Ii"]
  )
pal <- rev(RColorBrewer::brewer.pal(8, "RdYlBu"))
plot(marseille["LISA"], breaks = c(-8.5,-1.2,-0.7,-0.1,0,0.1,0.7,1.2,8.5), pal = pal)
```



f. Interprétez ce que vous voyez sur la carte.

```r
# On observe que les LISA sont très élevés sur quatre endroits principaux :
# - centre de Marseille au nord du Vieux Port
# - quartiers historiquement très pauvres comme la Belle-de-Mai;
# - au sud du Vieux Port - concentration d'Iris aux revenus élevés et entourés d'Iris
#   aux revenus médians élevés;
# - deux autres concentrations aux périphéries de la ville dans les 13eme et 11eme arrondissements.
```

g. Comme pour le I de Moran, il est nécessaire avant d'aller plus loin dans l'interprétation, de savoir si les Lisa calculés sont significativement différents de zéro.

Dans l'objet `mars_rev_lisa`, repérez la colonne correspondant à la pvaleur du test associé à la mesure des Lisa et placez-la dans une nouvelle variable du fond des Iris intitulée `LISA_PVAL`.

```r
marseille <- marseille %>%
  mutate(LISA_PVAL = mars_rev_lisa[,5])

# Il s'agit de la cinquième colonne intitulée `Pr(z != E(Ii))`.
```

h. Combien de LISA sont-ils significativement différents de zéro pour un niveau de confiance à 95% ?

```r
table(marseille$LISA_PVAL < 0.05)
```

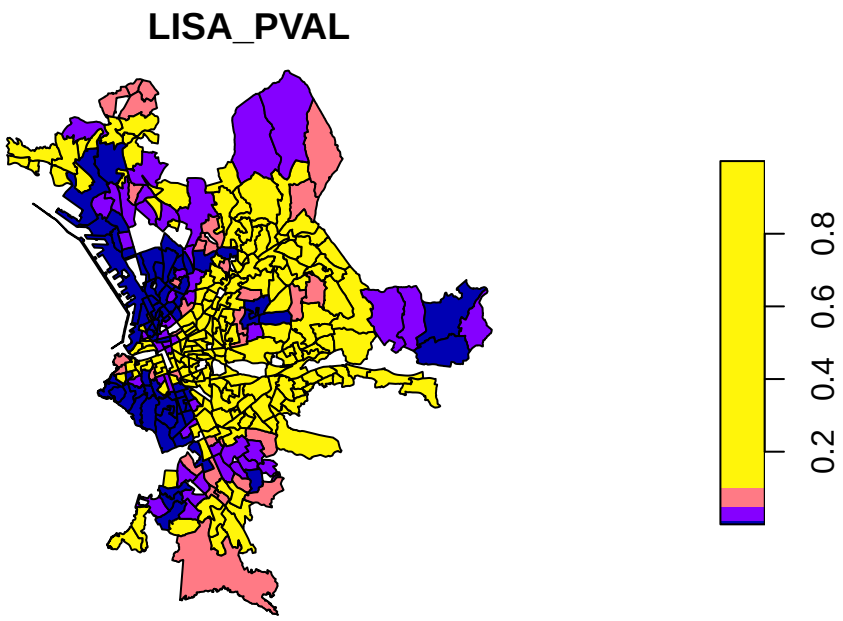
```
FALSE  TRUE
  222   114
```

```r
summary(marseille$LISA_PVAL)
```

```
     Min.   1st Qu.    Median      Mean   3rd Qu.      Max.
1.900e-07 1.881e-02 1.459e-01 3.081e-01 5.929e-01 9.963e-01
```

i. Représentez sur une carte la p-valeur des LISA en choisissant les bornes d'intervalles suivantes : 0,0.01,0.05,0.1,1.

```r
plot(marseille["LISA_PVAL"], breaks = c(0,0.01,0.05,0.1,1))
```



j. Les zones précédemment repérées sur la carte des LISA font-elles parties des zones où les LISA sont les plus significatifs ?

```r
# Oui, nous retrouvons avec les niveaux de significativité les plus élevés
#  (p-valeurs les plus faibles), les quatre zones précédemment décrites.
#
# > Remarque: En toute rigueur, il faudrait prendre en compte la multiplicité des tests dans
#  le clacul des p-valeurs. En effet, en réalisant 100 tests sur une distribution aléatoire
#  avec un niveau de confiance à 95%, le hasard fera qu'environ 5 tests passeront. Ici, on réalise
#  plus de 300 tests, donc, sans ajustement, le hasard produirait environ 15 tests positifs.
# L'ajustement peut être réalisé par exemple avec la méthode dite de Bonferroni.
# Pour en savoir plus, vous pouvez consulter le *Manuel d'Analyse Spatiale*, p.66-70.
```